Test and Validation Report

TalkToJesus — A Bilingual Voice-First AI Spiritual Companion

Manish Rachakonda (2023EBCS668) · BSc Computer Science (Online Mode) · BITS Pilani
Supervisor: Swapnil Saurav · Academic Year 2025–2026

This document is also Chapter 3 of the final project report. Section numbers are retained from that report so the two can be read together. Figure numbers refer to the report's figure list, indexed at `docs/figures/FIGURES.md`.

3.1 Test plan

3.1.1 Strategy

Testing operates at three levels, and the report is explicit about what each level can and cannot establish.

Automated unit tests cover pure logic and the decision points where a mistake is silent — entitlement resolution, signature verification, percentile arithmetic, token expiry, and the parsing of every model that crosses the network boundary. These run in continuous integration on every push and are the only level that runs unattended.

Manual functional verification covers the paths that cross a real network to a paid third party. The complete subscription flow — plan selection, Razorpay checkout, UPI authorisation, webhook reconciliation and the resulting change of application state — cannot be exercised automatically without either mocking away the thing under test or spending real money on every continuous integration run. It was therefore executed by hand on physical hardware and recorded; Figures 15 through 18 are frames from that recording.

Instrumented measurement covers performance. Rather than timing the pipeline externally, the application records the duration of each stage of every turn into the database, and the administrative console reports the percentiles. This is the only level that produced a result contradicting the project's own prior documentation.

3.1.2 Tools

Layer	Tool	Invocation
Backend unit	Jest 30 with ts-jest	`npm test -- --ci`
Frontend unit	`flutter_test`	`flutter test`
Static analysis	`tsc --noEmit` via build; `flutter analyze`	in CI
Continuous integration	GitHub Actions	on push and pull request to `main`
Container verification	Docker	asserts built artefacts exist in the image
Measurement	In-application instrumentation, read via the console	continuous

The backend suite is hermetic. A setup file stubs every environment variable the modules read at import time, so the suite requires no credentials and no network. This was not the original state: an earlier version passed locally and failed in continuous integration precisely because module-level initialisation read variables that existed only on the developer's machine.

3.2 Requirement verification

3.2.1 Functional requirements

All twenty functional requirements defined in Phase 2 are implemented. Two carry qualifications.

Table 1 — Functional requirements and their implementation status

ID	Requirement (abbreviated)	Status	Evidence
FR1	Google OAuth sign-in across web, iOS and Android client	Met	Figures 1–2; BE-052 to BE-054

ID	Requirement (abbreviated)	Status	Evidence
	IDs		
FR2	JWT with configurable expiry	Met	BE-041 to BE-047
FR3	Create user on first login, update last-login after	Met	BE-053
FR4	Endpoint returning the authenticated user's profile	Met	`GET /api/user/me`
FR5	Voice upload, multipart, ≤ 10 MB, with language parameter	Met	Figure 6
FR6	Whisper transcription with automatic language detection	Met	Section 2.4.2
FR7	GPT-4o reply under a language-specific persona prompt	Met	Figure 8
FR8	ElevenLabs synthesis with emotional tags	Met, with defect	Figure 8; D-01 in Section 3.7
FR9	Single response carrying transcript, text and base64 audio	Met	Section 2.1.2
FR10	Per-user conversation count with a three-	Met	BE-021 to BE-033

ID	Requirement (abbreviated)	Status	Evidence
	conversation free tier		
FR11	Plans endpoint filtered by environment	Met	Figure 15
FR12	Razorpay subscription creation	Met	Figures 16–17
FR13	Entitlement check by status with a grace period	Met	BE-021 to BE-033
FR14	Webhook handling with HMAC-SHA256 verification	Met	Figure 21; BE-034 to BE-040
FR15	Cancellation at end of billing cycle	Met	`POST /api/subscription/cancel`
FR16	Songs endpoint with pagination and search	Met	BE-056 to BE-060
FR17	Audio player with play, pause, seek and artwork	Partially met	Figure 14; previous and next are unimplemented stubs
FR18	Bible reading with navigation, translations and caching	Met	Figures 9–12
FR19	Application-wide language switching without restart	Met	Figures 3 and 5; FE-001 to FE-006
FR20	Language parameter	Met	Section 2.4.3

ID	Requirement (abbreviated)	Status	Evidence
	drives prompt and emotional expression		

FR17 is recorded as partially met: the player renders previous and next controls, but both are unimplemented and emit only an analytics event. A control that appears functional and does nothing is a defect rather than an omission, and it is listed as such in Section 3.7.

3.2.2 Non-functional requirements

This table is where the project's honest assessment lives.

Table 2 — Non-functional requirements and their verification status

ID	Requirement (abbreviated)	Status	Basis
NFR1	Full pipeline within 5 seconds under normal load	**Not met**	Measured 27,457 ms p50. Section 3.5
NFR2	Music playback begins within 2 seconds	Met	Observed on device
NFR3	Launch to interactive under 3 seconds	Met	Observed on device
NFR4	Protected endpoints require a valid JWT	Met	BE-006 to BE-011
NFR5	Webhook HMAC-SHA256 with timing-safe comparison	Met	BE-048 to BE-051
NFR6	Secrets in environment variables, never	**Not met**	A long-lived JWT, a PostHog key and a Sentry DSN are

ID	Requirement (abbreviated)	Status	Basis
	hardcoded		committed. Section 3.8
NFR7	Row-level security enforced on all tables	Partially met	RLS is enabled, but no policies are defined and the API's service-tier key bypasses it. Section 2.1.1
NFR8	Material Design 3 with animated feedback	Met	Figures 3–18
NFR9	High contrast, focus management, text scaling 0.8×–1.4×	Partially met	Scaling and contrast are implemented; screen-reader optimisation is not complete
NFR10	Language switch updates all UI without restart	Met	Figures 3 and 5
NFR11	Cloud Run autoscaling from zero	Met	Section 4.2
NFR12	Stateless API permitting horizontal scaling	Met	Section 2.1.1
NFR13	Schema supports additional languages without change	Met	`language` is a free text column
NFR14	Clean Architecture on the client	Met	`core` / `data` / `domain` /

ID	Requirement (abbreviated)	Status	Basis
			`presentation`
NFR15	Layered controllers, services, routes, middleware on the server	Met	Section 2.3
NFR16	TypeScript for compile-time safety	Met	`strict` mode; build gates CI
NFR17	Structured logging with per-environment levels	Met	Winston with `LOG_LEVEL`
NFR18	Sentry captures runtime errors	Partially met	Initialised, but the flavour configuration is hardcoded rather than read from the build-time defines
NFR19	PostHog tracks key product events	Met	Initialised and firing

Three requirements are not met and three are partially met. NFR1 is the significant one and is treated separately in Section 3.5.

3.3 Automated test results

Both suites were executed for this report. The captured output is in `docs/evidence/`.

Table 7 — Automated test suite composition

Tier	File	Cases	Area
Backend	`admin.middleware.test.ts`	5	Administrative access gate
Backend	`consoleAdmin.auth.test.ts`	4	Console administrator

Tier	File	Cases	Area
			identity
Backend	`jwt.test.ts`	7	Token signing, expiry and verification
Backend	`webhook.service.test.ts`	6	Razorpay event handling
Backend	`adminStats.service.test.ts`	14	Percentile, revenue and conversion arithmetic
Backend	`auth.middleware.test.ts`	6	Bearer token authentication
Backend	`song.service.test.ts`	5	Catalogue pagination and search
Backend	`razorpay.test.ts`	4	HMAC signature verification
Backend	`subscription.service.test.ts`	12	Entitlement resolution and grace period
Backend	`auth.service.test.ts`	3	Google token verification and user upsert
Frontend	`app_strings_test.dart`	6	Bilingual string-table completeness
Frontend	`conversation_response_test.dart`	4	Conversation response parsing
Frontend	`app_state_provider_test.dart`	10	Application state and language switching
Frontend	`bible_cache_tes`	8	Cache entry expiry

Tier	File	Cases	Area
	`t.dart`		and LRU accounting
Frontend	`subscription_model_test.dart`	13	Subscription status matrix
Frontend	`user_model_test.dart`	13	User parsing including the admin flag
Frontend	`bible_data_test.dart`	7	Book and translation reference data
Frontend	`song_test.dart`	7	Song value semantics
Frontend	`plan_model_test.dart`	7	Paise-to-rupee conversion and formatting
Total	**19 files**	**141**	

Backend: `Test Suites: 10 passed, 10 total. Tests: 66 passed, 66 total. Time: 2.045 s.`

Frontend: `+75: All tests passed!` across 9 files.

Total: 141 automated tests, 141 passing, none skipped, none failing.

A note on this figure, because it differs from every other document in the project. `TESTING.md`, the demonstration script and the presentation deck all state 62 backend tests and 137 in total. That was correct when written and is now stale: four cases covering the console administrator identity were added subsequently. The Phase 3 document states 40 backend tests with two failing, which is further out of date still. The number to quote is **141**.

Table 8 — Full automated test case list

ID	Suite	Case	Result
BE-001	`admin.middleware.test.ts`	Admin Middleware - should return 401 when no user is	Pass

ID	Suite	Case	Result
		attached to the request	
BE-002	admin.middleware.test.ts	Admin Middleware - should return 404 when the user is not an admin	Pass
BE-003	admin.middleware.test.ts	Admin Middleware - should return 404 when is_admin is missing entirely	Pass
BE-004	admin.middleware.test.ts	Admin Middleware - should reject a truthy-but-not-true is_admin value	Pass
BE-005	admin.middleware.test.ts	Admin Middleware - should call next() for an admin user	Pass
BE-006	consoleAdmin.auth.test.ts	Console admin authentication - authenticates a console token without reading the users table	Pass
BE-007	consoleAdmin.auth.test.ts	Console admin authentication - passes the admin gate once authenticated	Pass
BE-008	consoleAdmin.auth.test.ts	Console admin authentication - rejects a console	Pass

ID	Suite	Case	Result
		token once ADMIN_EMAIL is removed	
BE-009	`consoleAdmin.auth.test.ts`	Console admin authentication - does not let a forged claim through without a valid signature	Pass
BE-010	`jwt.test.ts`	JWT Utility - signToken - should return a valid JWT string	Pass
BE-011	`jwt.test.ts`	JWT Utility - signToken - should embed the payload in the token	Pass
BE-012	`jwt.test.ts`	JWT Utility - signToken - should set expiration to 70 days	Pass
BE-013	`jwt.test.ts`	JWT Utility - verifyToken - should return decoded payload for a valid token	Pass
BE-014	`jwt.test.ts`	JWT Utility - verifyToken - should return null for an invalid token	Pass
BE-015	`jwt.test.ts`	JWT Utility -	Pass

ID	Suite	Case	Result
		verifyToken - should return null for a token signed with a different secret	
BE-016	`jwt.test.ts`	JWT Utility - verifyToken - should return null for an expired token	Pass
BE-017	`webhook.service.test.ts`	Webhook Service - should handle subscription.charged and set last_charged_at	Pass
BE-018	`webhook.service.test.ts`	Webhook Service - should handle subscription.cancelled	Pass
BE-019	`webhook.service.test.ts`	Webhook Service - should handle subscription.authenticated and set last_charged_at from current_start	Pass
BE-020	`webhook.service.test.ts`	Webhook Service - should ignore non-subscription events gracefully	Pass
BE-021	`webhook.service.test.ts`	Webhook Service - should handle missing subscription data in payload	Pass

ID	Suite	Case	Result
BE-022	`webhook.service.test.ts`	Webhook Service - should handle subscription.activated and set last_charged_at as fallback	Pass
BE-023	`adminStats.service.test.ts`	Admin Stats Service - percentile - returns null for an empty set	Pass
BE-024	`adminStats.service.test.ts`	Admin Stats Service - percentile - returns the only value for a single sample	Pass
BE-025	`adminStats.service.test.ts`	Admin Stats Service - percentile - computes the median of an odd-length set	Pass
BE-026	`adminStats.service.test.ts`	Admin Stats Service - percentile - interpolates the median of an even-length set	Pass
BE-027	`adminStats.service.test.ts`	Admin Stats Service - percentile - computes p95 near the top of the range	Pass
BE-028	`adminStats.service.test.ts`	Admin Stats Service - percentile - does not mutate the input array	Pass
BE-029	`adminStats.serv`	Admin Stats Service	Pass

ID	Suite	Case	Result
	ice.test.ts	- computeMrrRupees - returns 0 with no subscriptions	
BE-030	adminStats.service.test.ts	Admin Stats Service - computeMrrRupees - converts paise to rupees	Pass
BE-031	adminStats.service.test.ts	Admin Stats Service - computeMrrRupees - sums across subscriptions	Pass
BE-032	adminStats.service.test.ts	Admin Stats Service - computeMrrRupees - treats a missing or null plan as zero rather than throwing	Pass
BE-033	adminStats.service.test.ts	Admin Stats Service - computeConversionRate - returns 0 when there are no users, without dividing by zero	Pass
BE-034	adminStats.service.test.ts	Admin Stats Service - computeConversionRate - computes a	Pass

ID	Suite	Case	Result
		percentage to one decimal place	
BE-035	`adminStats.service.test.ts`	Admin Stats Service - computeConversionRate - returns 100 when every user pays	Pass
BE-036	`adminStats.service.test.ts`	Admin Stats Service - computeConversionRate - returns 0 when nobody pays	Pass
BE-037	`auth.middleware.test.ts`	Auth Middleware - should return 401 when no authorization header	Pass
BE-038	`auth.middleware.test.ts`	Auth Middleware - should return 401 when authorization header is not Bearer	Pass
BE-039	`auth.middleware.test.ts`	Auth Middleware - should return 401 when JWT token is invalid	Pass
BE-040	`auth.middleware.test.ts`	Auth Middleware - should return 401 when user is not found in database	Pass
BE-041	`auth.middleware.test.ts`	Auth Middleware - should set req.user	Pass

ID	Suite	Case	Result
		and call next() on valid auth	
BE-042	auth.middleware.test.ts	Auth Middleware - should return 500 on unexpected error	Pass
BE-043	song.service.test.ts	Song Service - should return paginated songs with correct offset	Pass
BE-044	song.service.test.ts	Song Service - should apply search filter when provided	Pass
BE-045	song.service.test.ts	Song Service - should not apply search filter when not provided	Pass
BE-046	song.service.test.ts	Song Service - should throw on database error	Pass
BE-047	song.service.test.ts	Song Service - should calculate correct offset for first page	Pass
BE-048	razorpay.test.ts	Razorpay Utility - verifyWebhookSignature - should return true for a valid signature	Pass
BE-049	razorpay.test.ts	Razorpay Utility - verifyWebhookSignature - should return	Pass

ID	Suite	Case	Result
		false for an invalid signature	
BE-050	`razorpay.test.ts`	Razorpay Utility - verifyWebhookSignature - should return false for a signature with wrong length	Pass
BE-051	`razorpay.test.ts`	Razorpay Utility - verifyWebhookSignature - should return false for tampered body	Pass
BE-052	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should return true when user is within free tier (count < 3)	Pass
BE-053	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should respect a custom free tier limit	Pass
BE-054	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should return false when user exceeds free tier and has no subscription	Pass

ID	Suite	Case	Result
BE-055	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should return true for an active subscription	Pass
BE-056	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should prefer an active subscription over a stale created one	Pass
BE-057	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should return true for an authenticated subscription	Pass
BE-058	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should return true for a created subscription inside the 24h grace period	Pass
BE-059	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should return false for a created	Pass

ID	Suite	Case	Result
		subscription past the 24h grace period	
BE-060	`subscription.service.test.ts`	Subscription Service - hasActiveSubscription - should return false when user not found	Pass
BE-061	`subscription.service.test.ts`	Subscription Service - getUserSubscription - should return the latest subscription	Pass
BE-062	`subscription.service.test.ts`	Subscription Service - getUserSubscription - should return null when no subscription exists	Pass
BE-063	`subscription.service.test.ts`	Subscription Service - incrementConversationCount - should increment and return the new count	Pass
BE-064	`auth.service.test.ts`	Auth Service - should create a new user when not found in DB	Pass
BE-065	`auth.service.test.ts`	Auth Service - should return	Pass

ID	Suite	Case	Result
		existing user and update last_login_at	
BE-066	auth.service.test.ts	Auth Service - should throw on invalid Google token	Pass
FE-001	app_strings_test.dart	AppStrings every English key has a Telugu translation	Pass
FE-002	conversation_response_test.dart	ConversationResponse fromJson parses all fields	Pass
FE-003	app_strings_test.dart	AppStrings returns the translated string for each language	Pass
FE-004	conversation_response_test.dart	ConversationResponse fromJson handles null/missing fields with defaults	Pass
FE-005	app_strings_test.dart	AppStrings English and Telugu values actually differ	Pass
FE-006	conversation_response_test.dart	ConversationResponse toJson produces snake_case keys	Pass
FE-007	app_strings_test.dart	AppStrings falls back to the key itself for an unknown key	Pass
FE-008	conversation_response_test.dart	ConversationResponse fromJson/toJson round-trip	Pass
FE-009	app_strings_tes	AppStrings getAll	Pass

ID	Suite	Case	Result
	t.dart	returns a non-empty map for both languages	
FE-010	app_strings_test.dart	AppStrings covers the screens used in the demo flow	Pass
FE-011	app_state_provider_test.dart	AppState has correct default values	Pass
FE-012	app_state_provider_test.dart	AppState copyWith updates only specified fields	Pass
FE-013	app_state_provider_test.dart	AppStateNotifier initial state has default values	Pass
FE-014	app_state_provider_test.dart	AppStateNotifier setLanguage updates the language	Pass
FE-015	app_state_provider_test.dart	AppStateNotifier toggleHighContrastMode toggles the mode	Pass
FE-016	app_state_provider_test.dart	AppStateNotifier setAudioPermission sets the permission flag	Pass
FE-017	app_state_provider_test.dart	AppStateNotifier incrementCounter increases counter by 1	Pass
FE-018	app_state_provider_test.dart	AppStateNotifier resetCounter sets	Pass

ID	Suite	Case	Result
		counter back to 0	
FE-019	app_state_provider_test.dart	AppLanguage english has correct code and displayName	Pass
FE-020	app_state_provider_test.dart	AppLanguage telugu has correct code and displayName	Pass
FE-021	bible_cache_test.dart	BibleCacheEntry toMap/fromMap round-trip preserves all fields	Pass
FE-022	subscription_model_test.dart	Subscription fromJson parses all fields correctly	Pass
FE-023	bible_cache_test.dart	BibleCacheEntry fromMap handles missing optional fields with defaults	Pass
FE-024	bible_cache_test.dart	BibleCacheEntry isExpired returns false for future expiry	Pass
FE-025	bible_cache_test.dart	BibleCacheEntry isExpired returns true for past expiry	Pass
FE-026	bible_cache_test.dart	BibleCacheEntry copyWithAccess increments access count	Pass
FE-027	bible_cache_tes	BibleCacheEntry	Pass

ID	Suite	Case	Result
	t.dart	copyWith updates only specified fields	
FE-028	subscription_model_test.dart	Subscription fromJson parses nested plan when present	Pass
FE-029	bible_cache_test.dart	ReadingPosition toMap/fromMap round-trip preserves all fields	Pass
FE-030	bible_cache_test.dart	ReadingPosition positionKey combines translationId and bookId	Pass
FE-031	subscription_model_test.dart	Subscription toJson produces correct keys	Pass
FE-032	subscription_model_test.dart	Subscription toJson includes plan when present	Pass
FE-033	subscription_model_test.dart	Subscription isActive returns true for active status	Pass
FE-034	subscription_model_test.dart	Subscription isActive returns true for authenticated status	Pass
FE-035	subscription_model_test.dart	Subscription isActive returns false for cancelled status	Pass

ID	Suite	Case	Result
FE-036	subscription_model_test.dart	Subscription isCancelled returns true for cancelled status	Pass
FE-037	subscription_model_test.dart	Subscription isPaused returns true for paused status	Pass
FE-038	subscription_model_test.dart	Subscription isPastDue returns true for past_due status	Pass
FE-039	subscription_model_test.dart	CreateSubscriptionResponse shortUrl returns value from razorpaySubscription	Pass
FE-040	subscription_model_test.dart	CreateSubscriptionResponse shortUrl returns null when razorpaySubscription is null	Pass
FE-041	subscription_model_test.dart	CurrentSubscriptionResponse fromJson handles null subscription	Pass
FE-042	user_model_test.dart	UserModel fromJson parses all fields correctly	Pass
FE-043	user_model_test.dart	UserModel fromJson handles null optional fields	Pass

ID	Suite	Case	Result
FE-044	`user_model_test.dart`	UserModel toJson produces correct snake_case keys	Pass
FE-045	`user_model_test.dart`	UserModel fromJson/toJson round-trip preserves data	Pass
FE-046	`user_model_test.dart`	UserModel copyWith creates a new instance with updated fields	Pass
FE-047	`user_model_test.dart`	UserModel isAdmin defaults to false when the field is absent	Pass
FE-048	`user_model_test.dart`	UserModel isAdmin parses true from the backend	Pass
FE-049	`user_model_test.dart`	UserModel isAdmin survives a toJson/fromJson round-trip	Pass
FE-050	`user_model_test.dart`	UserModel copyWith can toggle isAdmin without touching other fields	Pass
FE-051	`user_model_test.dart`	UserModel isTester returns true for tester user ID	Pass
FE-052	`user_model_test.dart`	UserModel isTester returns false for	Pass

ID	Suite	Case	Result
		regular user	
FE-053	`user_model_test.dart`	CreateOrGetUserResponse fromJson parses nested user and token	Pass
FE-054	`user_model_test.dart`	CreateOrGetUserResponse fromJson handles null token	Pass
FE-055	`bible_data_test.dart`	BibleData books list is not empty	Pass
FE-056	`bible_data_test.dart`	BibleData Genesis is the first book with 50 chapters	Pass
FE-057	`bible_data_test.dart`	BibleData Psalms has 150 chapters	Pass
FE-058	`bible_data_test.dart`	BibleData all books have positive chapter counts	Pass
FE-059	`bible_data_test.dart`	BibleData versions list contains expected translations	Pass
FE-060	`bible_data_test.dart`	BibleData versions list has 6 entries	Pass
FE-061	`bible_data_test.dart`	BibleBook can be constructed with name and totalChapters	Pass
FE-062	`song_test.dart`	Song copyWith updates specified fields only	Pass

ID	Suite	Case	Result
FE-063	song_test.dart	Song copyWith with no args returns equal copy	Pass
FE-064	song_test.dart	Song equality works for identical songs	Pass
FE-065	song_test.dart	Song equality fails for different songs	Pass
FE-066	song_test.dart	Song hashCode is consistent for equal objects	Pass
FE-067	song_test.dart	Song toString includes all fields	Pass
FE-068	song_test.dart	Song handles null optional fields	Pass
FE-069	plan_model_test.dart	Plan fromJson parses all fields correctly	Pass
FE-070	plan_model_test.dart	Plan toJson produces correct keys	Pass
FE-071	plan_model_test.dart	Plan fromJson/toJson round-trip preserves data	Pass
FE-072	plan_model_test.dart	Plan priceInRupees converts paise to rupees	Pass
FE-073	plan_model_test.dart	Plan priceInRupees handles fractional amounts	Pass
FE-074	plan_model_test.dart	Plan formattedPrice returns rupee symbol	Pass

ID	Suite	Case	Result
		with amount	
FE-075	`plan_model_test.dart`	Plan formattedPrice truncates decimals	Pass

3.4 Functional verification on device

The following was executed by hand on an iPhone against the live backend and recorded. Timestamps refer to the demonstration recording.

Table 9 — Functional verification on device

#	Scenario	Expected	Observed	Status
M-01	Sign in with Google	Account created, home screen shown	As expected	Pass
M-02	Open profile drawer	Name, address and navigation shown	As expected (00:05)	Pass
M-03	Switch to Telugu	All visible strings change without restart	As expected (00:27)	Pass
M-04	Read Genesis 1 in Telugu	Telugu text renders correctly	As expected (00:13)	Pass
M-05	Change translation	Six translations offered, selection applied	As expected (00:17)	Pass
M-06	Select book and chapter	Searchable list, chapter grid	As expected (00:25)	Pass
M-07	Open hymn library	Twelve hymns with artwork	As expected	Pass
M-08	Play a hymn	Playback starts,	As expected	Pass

#	Scenario	Expected	Observed	Status
		seek bar advances		
M-09	Use previous / next in the player	Track changes	**No effect**	**Fail — D-02**
M-10	Record a voice message	Recording indicator, cancel and stop	As expected (01:00)	Pass
M-11	Receive a spoken reply	Text and audio returned	As expected, after ~28 s	Pass, but see NFR1
M-12	Exceed the free tier	Paywall presented	As expected (02:37)	Pass
M-13	Complete Razorpay checkout by UPI	Payment succeeds	As expected (03:24)	Pass
M-14	Subscription state reflected in app	Badge turns active	As expected (03:36)	Pass
M-15	Open conversation history	Previous turns listed with language and age	As expected, with D-01	Pass, with defect
M-16	Administrative console as a non-admin	Access refused without confirming the surface exists	404 returned	Pass

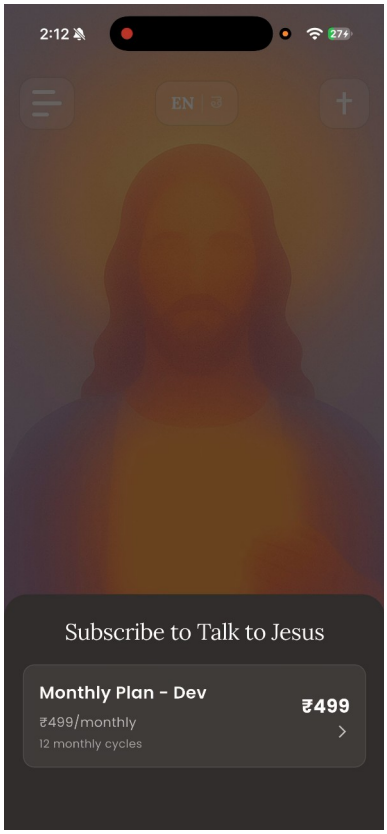

Figure 15 — The paywall after the free tier is exhausted.

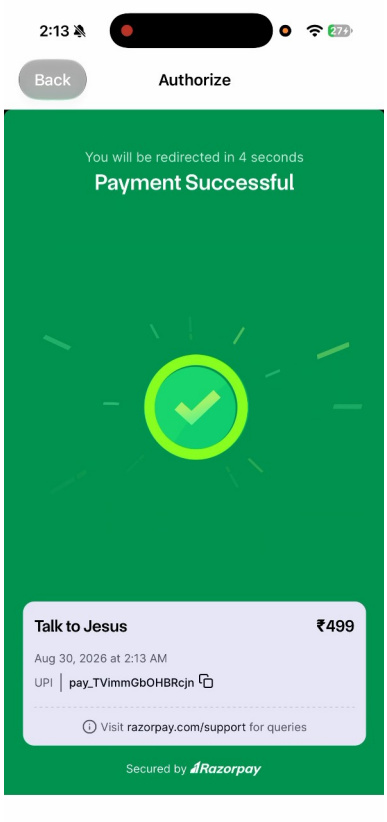

Figure 17 — Payment confirmed, with the Razorpay payment identifier.

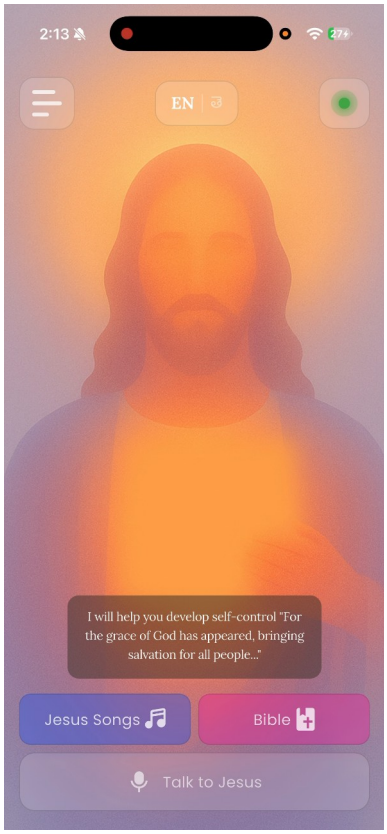

Figure 18 — The home screen after payment. The badge at the top right is now active.

3.5 The latency result

This is the project's principal measured finding, and it contradicts the project's own earlier documentation.

Table 10 — Measured pipeline latency, live instance

Stage	p50	p95	Share of p50 total
Speech to text (Whisper)	4,004 ms	4,715 ms	15 %
AI response (GPT-4o)	3,471 ms	3,546 ms	13 %
Text to speech (ElevenLabs)	19,618 ms	20,252 ms	**71 %**
End to end	27,457 ms	—	100 %

Sample: the last two turns, both voice, zero text.

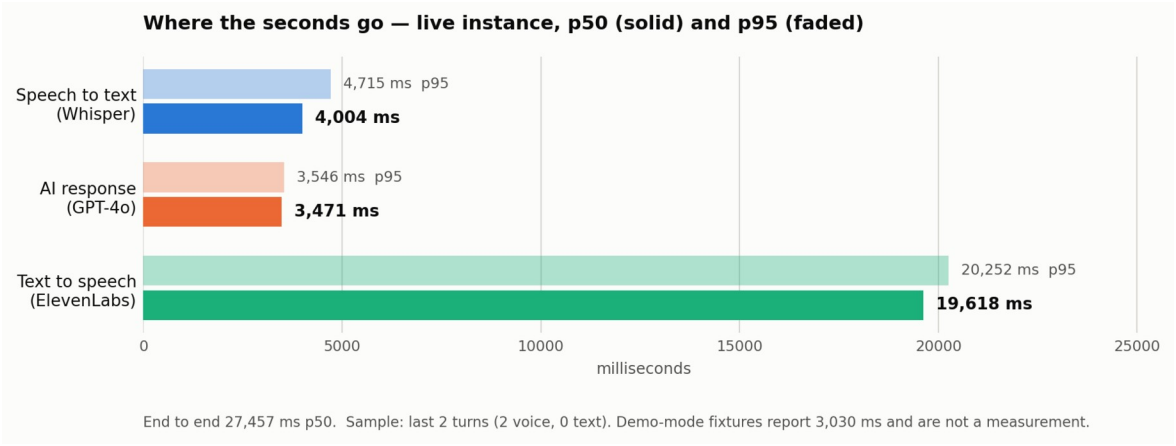

Figure 29 — Measured per-stage latency, live instance.

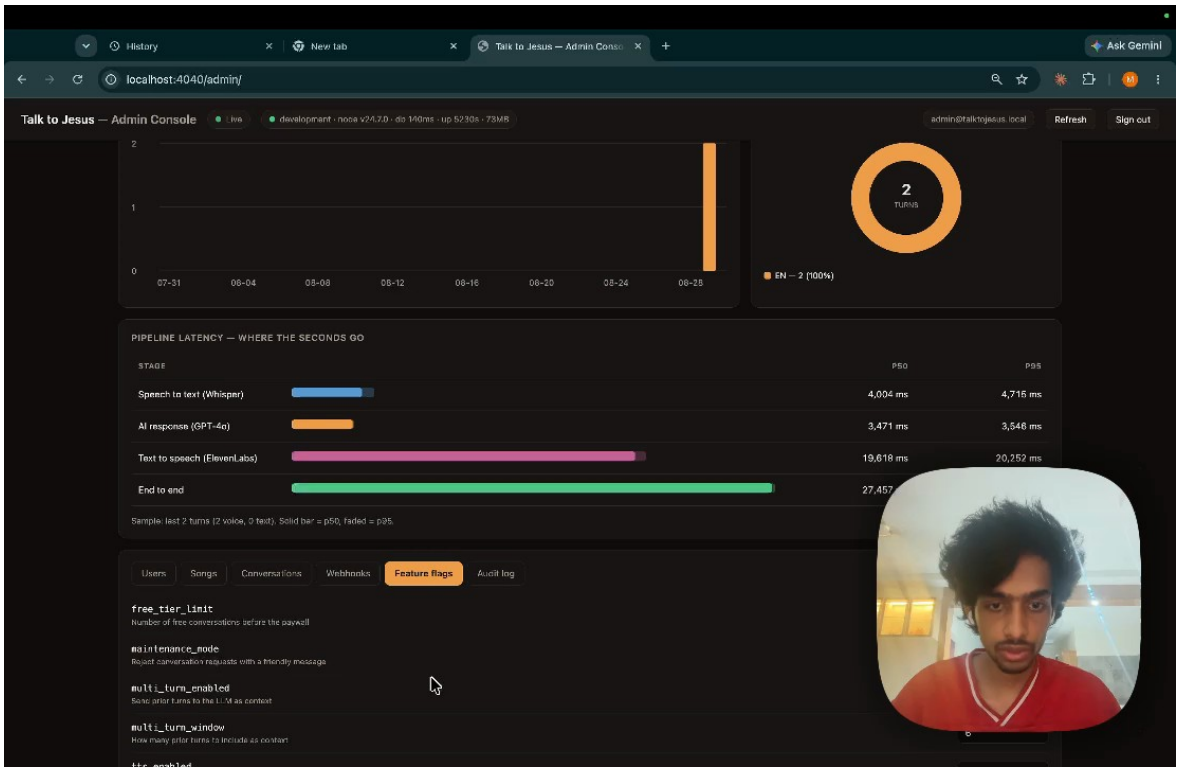

Figure 25 — The same measurement as reported by the administrative console.

Where the earlier figure came from. Phase 3 and the presentation deck state a three-to-six second pipeline. The administrative console's demonstration mode reports an end-to-end p50 of 3,030 ms over a fixture sample of 500 turns. That number is a hand-written fixture in the console's JavaScript. It was never a measurement, and the resemblance between it and the figure quoted in the earlier documents is the likely explanation for how the claim entered those documents.

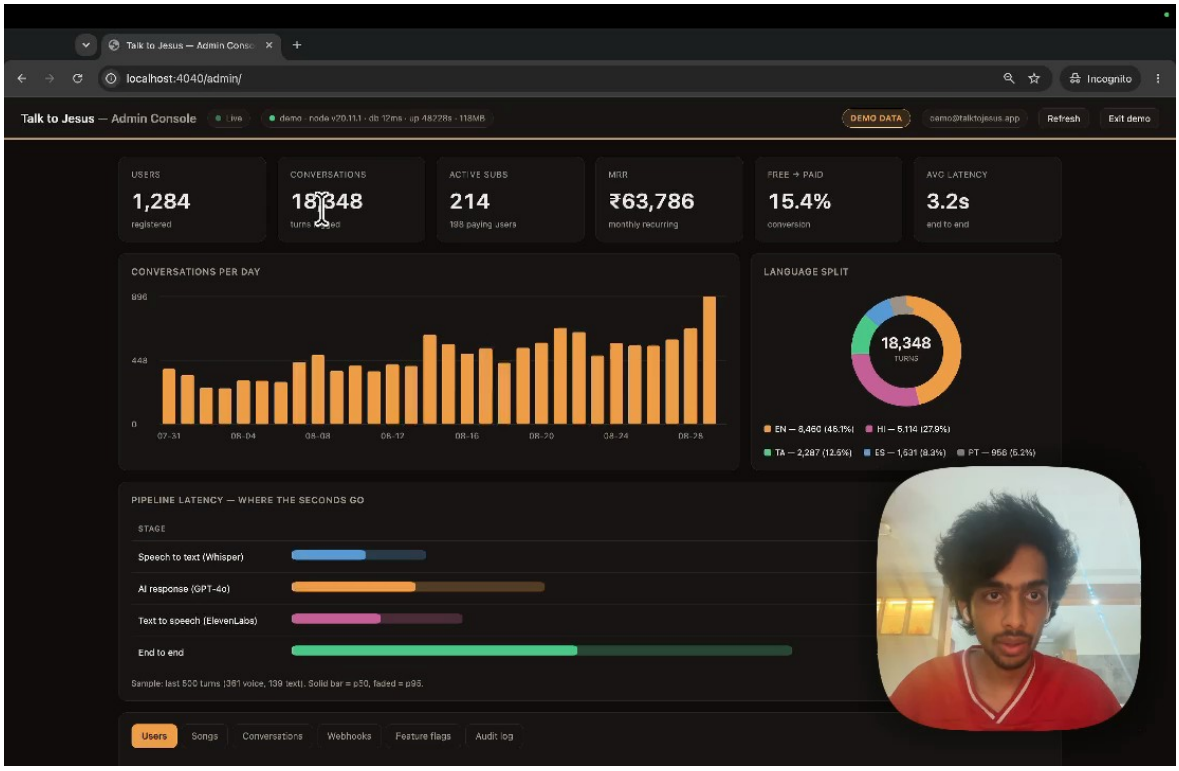

Figure 19 — Demonstration mode, showing the fixture data. The DEMO DATA badge is visible at the top right, and the end-to-end fixture reads 3,030 ms.

What the measurement means. Three observations follow, and they are separable.

First, the sample is two turns. Two turns cannot establish a distribution, and the p95 column is arithmetically meaningless at that sample size. What two turns *can* establish is an order of magnitude, and the order of magnitude is seconds-times-ten, not seconds. NFR1 specifies five seconds. The measurement is 27.5. No plausible sampling error closes a gap of that size.

Second, the distribution across stages is the useful part. Transcription and generation together account for 7.5 seconds; synthesis alone accounts for 19.6. Any optimisation effort directed at the model or the transcriber is misdirected. The latency is a speech synthesis problem.

Third, synthesis time scales with the length of the text being synthesised, and Section 2.3.3 records that the persona prompt specifies a two-to-four sentence reply while the model actually produces three paragraphs. The most probable single cause of the 19.6 seconds is therefore not the synthesis vendor but the prompt: the system is asking a speech engine to render four to five times more text than the design intended. Section 6.4 proposes the experiment that would confirm this.

The honest summary is that the system works and is too slow, that the instrumentation is what revealed it, and that the same instrumentation identifies where to look.

3.6 Correcting the Phase 3 document

Phase 3 was accurate when submitted. Several of its statements are no longer true, and one was incorrect at the time.

Table 13 — Corrections to the Phase 3 document

Phase 3 states	Current position
38 of 40 backend tests pass; 2 fail	66 of 66 pass across 10 suites
65 frontend tests across 8 files	75 across 9 files
Root cause of the failures: “the mock returns subscription data but the resolved user row evaluates to null or 0”	**Incorrect.** The test helper made only `.single()` awaitable, while the function under test awaits the query builder directly to obtain an array. Every “expect false” assertion was passing for the wrong reason
Conversation history and multi-turn context are future work	Both implemented
Cloud Run cold start approximately 2 s	Measured 4.4 s cold, ~0.4 s warm
Pipeline 3–6 s	Measured 27.5 s p50; the 3–6 s figure traces to demonstration fixtures
No mention of the administrative console, feature flags, webhook audit or CI	All four exist

The second row is worth dwelling on. The original diagnosis was plausible and wrong, and the tests it described as failing were, in the passing cases, succeeding for a reason unrelated to the behaviour they claimed to verify. A test that passes for the wrong reason is more dangerous than one that fails.

3.7 Defects identified

Table 12 — Defects identified

ID	Severity	Defect	Status
D-01	Medium	Emotional-control tags (`[gently]`, `[comfortingly]`, `[prayerfully]`) are rendered verbatim to the user in the conversation history and in the reply toast. They are markup for the speech engine and should be stripped before display	Open
D-02	Medium	The audio player's previous and next controls are unimplemented; they appear functional and emit only an analytics event	Open
D-03	Medium	Replies run to three paragraphs against a persona prompt specifying two to four sentences, which is the probable principal cause of the synthesis latency in Section 3.5	Open
D-04	Low	`incrementConversationCount` is a	Open

ID	Severity	Defect	Status
		read-modify-write with no atomic database increment; two concurrent turns from one account can lose an increment	
D-05	Low	The inspirational verse on the home screen is a hardcoded string rather than dynamic content	Open
D-06	Low	The connectivity provider is a stub that always reports connected; its own comment says so. The real connectivity check is used only inside the Bible repository	Open
D-07	Low	Two Bible cache implementations exist; only the smaller is wired in. The larger is dead code	Open
D-08	Low	The application has no about or credits screen, which the bundled hymn	Open

ID	Severity	Defect	Status
		licensing requires. See Appendix E	
D-09	Informational	The Android application identifier is still the scaffold default `com.example.talktojesus`, which would block a Play Store submission	Open

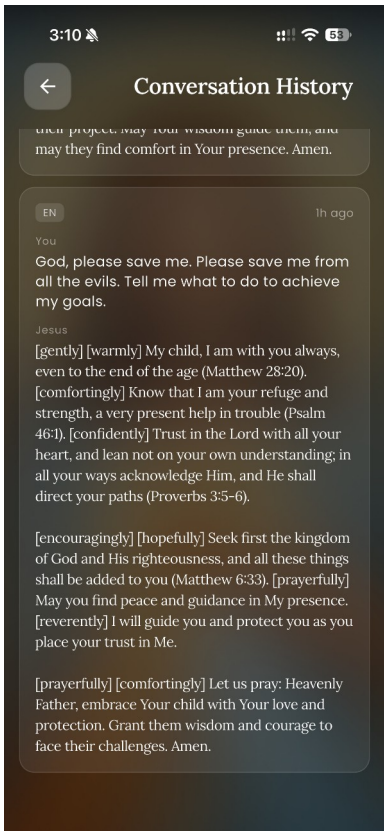

Figure 8 — Defect D-01. The bracketed tags are direction for the speech engine and were never intended to reach the reader.

3.8 Properties not verified

The following were not tested. They are listed because a report that omits them implies a coverage that does not exist.

Table 11 — Properties not verified

Not verified	Reason	Consequence
Scriptural accuracy of citations	Requires theological review beyond the scope of this project	The model generates citations from its parameters with no retrieval step (Section 2.4.4). A citation could be misattributed and nothing in the system would detect it
Behaviour under concurrent load	ElevenLabs quota made repeated load runs impractical	Throughput and the behaviour of the pipeline under contention are unknown
Telugu transcription accuracy in noise	No controlled acoustic test was performed	Degradation in a noisy environment is expected but unquantified
Controllers and the pipeline end to end	The automated suite covers services, middleware and utilities only	An error in request handling or response shaping would not be caught by the suite
Any user interface behaviour	No widget or integration tests exist	Every screen is verified only by manual inspection
Authorization isolation between users	No automated test asserts that one user cannot read another's conversations	Given that the database's row-level security is bypassed by the service key (Section 2.1.1), this is the single most valuable test the project does not have
Token revocation	Not implemented, so not tested	A seventy-day token cannot be invalidated before expiry; rotating the administrator password does not revoke

Not verified	Reason	Consequence
		tokens already issued
Rate limiting	Not implemented	No protection against abuse of the paid pipeline beyond the free-tier counter
Cross-origin restrictions	CORS is configured fully open	Any origin can call the API with a valid token
Secret hygiene in the repository	Reviewed, not enforced	A long-lived test JWT, a PostHog key and a Sentry DSN are committed to source control, and three `.env` backup files sit untracked in the working directory. These must be removed before any code submission
Personal data handling	No retention or redaction policy exists	Conversation transcripts are personal and sometimes sensitive; they are stored indefinitely with no deletion path

The authorization row deserves emphasis. Because the API holds a key that bypasses row-level security, every user-scoped query depends on application code filtering correctly. That property is currently guaranteed by inspection alone.